Algorithm Analysis –

Algorithm analysis is the process of measuring how efficient is an algorithm in terms of —
time complexity – time taken to execute.
space complexity – memory used during execution.
The efficiency is measured as the input size (n) increases.

Time Complexity

Time complexity represents the amount of time an algorithm takes to complete as the input size (n) increases.
→ It does not measure actual execution time (seconds); instead it measures how the number of operation grows.

Space Complexity

Space complexity is the total memory required by an algorithm during execution.
It includes: input ~~variables~~, memory, variables, auxiliary memory, recursive stack memory.

Asymptotic Notations describes the growth rate of algorithms.

- Big O (O) - represents the upper bound (worst case performance).
- Omega (Ω) - represents the lower bound (best - case performance)
- Theta (Θ) - represents the tight bound (average-case performance)

Best case - Minimum no: of operations. Represented by Big-O notation : $O(n)$

Average case - Average no: of operations over all inputs. Represented by Theta notation : $\Theta(n)$.

Worst case - Maximum number of operations. Represented by Omega notation : $\Omega(n)$.

Rules to Calculate complexity

Rule 1 : Ignore constants
Constant operations take the same time regardless of input size.

Ex - int a=10;
cout << a;

T.C $\Rightarrow O(1)$

Rule 2: Add the complexities if Sequential Statements

Ex- for(int i=0; i<n; i++)
    cout << i;
  for(int j=0; j<n; j++)
    cout << j;

T.C ⇒ O(n) + O(n) = O(2n)
(Ignore constants) ⇒ O(n) : T.C.

Rule 3: Multiple complexities if nested loops

Ex - for(int i=0; i<n; i++) {
    for(int j=0; j<n; j++) {
      cout << i << j;
    }
  }

T.C ⇒ O(n*n) = O(n²)

Rule 4: Drop lower order terms

Ex- O(n²+n+10) = O(n²)

Because as n becomes very large, n² dominates the other terms.

Rule 5: Ignore constants in multiplication

Ex - O(5n) = O(n)

Rule 6: Loop depends on Input size

Count how many times the loop runs.

Ex- for (int i=0; i<n; i++)
  Runs n times ⟶ O(n)

for(int i=0; i<n; i*=2)
Runs $\log_2 n$ times → $O(\log n)$

Rule 7: Different variables remain separate
Ex - for(int i=0; i<n; i++) {
... }
for(int j=0; j<m; j++){
... }
T.C ⇒ $O(n+m)$

Ex - for (int i=0; i<n; i++) {
for(int j=0; j<m; j++) {
...
}
}
TC ⇒ $O(n*m)$

Rule 8 : Conditional statements
count only the worst-case branch

Time Complexity of Logarithmic Loops
When the loop variable is multiplied or divided by a constant in each iteration, the complexity is logarithmic.

Space Complexity
It consists of:
- Input space
- Auxiliary space (extra memory used)
- Recursion stack

- Time complexity measures how execution time grows with input size.
- Space complexity measures additional memory used by an algorithm.
- Big-O (O) represents the worst case complexity.
- Thata (Θ) represents the average case complexity.
- Omiga (Ω) represents the best case complexity.
- In complexity analysis, ignore constants and lower-order terms.
- Single loops are typically $O(n)$, nested loops are $O(n^2)$, and loop the repeatedly double or halve the iterator are $O(\log n)$.

Section-A

Module 1 - Complexity Analysis

In all the patterns, the two nested loops are ~~present~~ required for the output. Therefore, the time complexity for all the patterns is $O(n^2)$.
Only loop variables are used, so the space complexity is $O(1)$.

Module 2 - Complexity Analysis

1. Fraud Transaction Detection (Linear Search)

Time Complexity

- Best Case : $O(1)$
- Average Case : $O(n)$
- Worst Case : $O(n)$

Space Complexity : $O(1)$

Linear search checks each transaction ID one by one until the required ID is found or the entire list ~~list~~ is searched. In worst case, every element is examined, giving $O(n)$ time complexity. Only a few variables are used, so space complexity is $O(1)$.

Optimized Approach - If the transaction IDs are sorted, Binary search can be used, reducing the search time to $O(\log n)$.

2. Server Log Search (Binary Search)

Time Complexity

- Best Case: $O(1)$
- Average case: $O(\log n)$
- Worst case: $O(\log n)$

Space complexity – $O(1)$

Binary Search repeatedly divided the sorted array into two halves, eliminating half of the remaining elements in each step making the time complexity $O(\log n)$.

Optimized Approach – It is already one of the most efficient searching algos.

3. Sort Employee Salaries (Bubble Sort)

Time Complexity

- Best Case: $O(n)$
- Avg case: $O(n^2)$
- Worst case: $O(n^2)$

Space Complexity – $O(1)$

It repeatedly compares and swaps adjacent elements until the array is sorted. In avg and worst case multi swaps are required, so $T.C = O(n^2)$. Bubble Sorting is in-place, therefore, $S.C = O(1)$.

For larges datasets, MergeSort or QuickSort is preferred since they have $O(n \log n)$ avg time complexity.

4. Sort Website Response Time (Insertion Sort)

Time Complexity
- Best Case : $O(n)$
- Avg Case : $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity - $O(1)$

It inserts each element into its correct position within the already sorted portion. It requires many shifts in the worst case, giving $O(n^2)$ time complexity.

Optimized Approach - for large datasets, MergeSort or QuickSort are preferred.

5. Prioritize Bug Reports (Selection Sort)

Time Complexity
- Best Case: $O(n^2)$
- Avg Case: $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity - $O(1)$

It repeatedly finds the minimum element from unsorted part of the array and places it in its correct position. Since it performs same no. of comparisons in all the cases, time complexity remains $O(n^2)$. And space complexity is $O(1)$ since it's in-place sorting.

For optimized Approach, Heap, Merge or Quick sort provides better performance reducing T.C to $O(n \log n)$

Module 3 - Complexity Analysis

Merge Sort (Question 1 and 2)

Time Complexity : $O(n \log n)$
Space Complexity : $O(n)$
Merge Sort recursively divides the array into two halves until single element remains
$\therefore$ TC $\rightarrow O(n \log n)$.
Additional temporary array is required during merging, $\therefore$ SC $\rightarrow O(n)$.

Quick Sort (Question 1 and 2)

Time Complexity : $O(n \log n)$ - Avg case
$O(n^2)$ - Worst case
Space Complexity : $O(\log n)$
It partitions the array around a pivot and recursively sorts the subarrays. On avg, the partitions are balanced, giving $O(n \log n)$ but in worst case (already sorted array), the complexity becomes $O(n^2)$.

Optimized Approach - Choosing a random pivot significantly reduces the chance of the worst case.

Heap Sort (Question 1 and 2)

Time Complexity: $O(n \log n)$
Space Complexity: $O(1)$
It first builds a heap in $O(n)$ time and then repeatedly removes the maximum element, each taking $O(\log n)$, making the time complexity $O(n \log n)$.
Since, the sorting is done in place, only constant extra memory is required.

Optimized Approach - It is already an efficient in-place sorting algorithm.

Section-B

Problem 1 - Print no.s from 1 to N

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cout << i << " ";
    }
    return 0;
}
```

Time Complexity — O(n)
Space Complexity — O(1)

The program uses a single loop that runs from 1 to n. Since each number is printed exactly once, the no. of operations grow linearly with n resulting in O(n) time complexity. Only a few integer variables are used, so the space complexity is O(1).

Problem 2 - Sum of first n natural numbers

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    long long sum = 0;

    for( int i = 1; i <= n; i++) {
        sum += i;
    }
    cout << sum;
    return 0;
}
```

Time Complexity — O(n)
Space complexity — O(1)

The loop executes n times and adds each natural number to the sum. Therefore, the time complexity is O(n) and only the variables n, i and sum are used, so the space complexity is O(1).

For optimized approach, we can used mathematical formula : Sum = (n(n+1))/2 which reduces the time complexity to O(1) and space complexity remains O(1).

Problem 3 - Find A^B.

```
#include<bits/stdc++.h>
using namespace std;

int main(){
    int A,B;
    cin >> A >> B;
    long long result = 1;
    for(int i=1; i<=B; i++){
        result = result * A;
    }
    cout << result;
    return 0;
}
```

Time Complexity - O(B)
Space Complexity - O(1)

The program calculates A^B by multiplying A with the result B times using a loop. Therefore, the time complexity is O(B), since loop runs B times. Since the constant amount of memory is used, so the space complexity is O(1).

For optimized approach, Binary Exponentation can be used which computes A^B in O(log B) time by repeatedly squaring the base and halving the exponent.

```cpp
#include <iostream>
using namespace std;

int main() {
    int N;
    cin >> N;

    for(int i = 1; i <= N; i++) {
        cout << i << " ";
    }

    return 0;
}
```

```
5
1 2 3 4 5
```

```cpp
#include <iostream>
using namespace std;

int main() {
    int N;
    cin >> N;

    long long sum = 0;

    for(int i = 1; i <= N; i++) {
        sum += i;
    }

    cout << sum;

    return 0;
}
```

```
5
15
```

```cpp
#include <iostream>
using namespace std;

int main() {
    int A, B;
    cin >> A >> B;

    long long result = 1;

    for(int i = 1; i <= B; i++) {
        result = result * A;
    }

    cout << result;

    return 0;
}
```

```
2 5
32
```